ใบโจทย์: Array in C

ชื่อ ..... ชั้น/กลุ่ม ..... วันที่ .....

เกณฑ์คะแนน: 5 ข้อ ข้อละ 20 คะแนน รวม 100 คะแนน

เติมเฉพาะ TODO ใน array_lab.c ไม่เปลี่ยนชื่อฟังก์ชัน พารามิเตอร์ หรือ TEST RUNNER
แต่ละข้อมี 10 เทส (รวม edge cases) เทสละ 2 คะแนน ไม่ผ่านเทสนั้นได้ 0 คะแนน
Input คือค่าที่ตัวตรวจส่งให้ฟังก์ชัน ไม่ต้องเขียน scanf; Expected คือค่าที่ต้องคืนหรือเขียนลงอาเรย์

01 หาค่าสูงสุดและต่ำสุด | 20 คะแนน

```
void find_max_min(const int a[], int n, int result[2])
```

n >= 1; ห้ามเปลี่ยนค่าใน a และต้องตรวจครบทุกสมาชิก
เขียนค่าสูงสุดใน result[0] และต่ำสุดใน result[1]
ครอบคลุม: ค่าติดลบ ค่าซ้ำ ศูนย์ สมาชิกเดียว และค่าเล็ก/ใหญ่สุดของ int

Example Input	Expected Result
`{12, 7, 25, 3, 18}`	`result = {25, 3}`
`{-8, -2, -10, -4}`	`result = {-2, -10}`
`{-7}`	`result = {-7, -7}`
`{INT_MIN, 0, INT_MAX}`	`result = {INT_MAX, INT_MIN}`

รัน: ./array_lab maxmin | 10 เทส x 2 คะแนน = 20 คะแนน

02 หาค่าเฉลี่ยของสมาชิก | 20 คะแนน

```
double find_average(const int a[], int n)
```

n >= 1; ห้ามเปลี่ยน a; return ค่าเฉลี่ยเป็น double ที่เก็บทศนิยมได้
ยอมรับความคลาดเคลื่อนไม่เกิน 0.000001; ผลรวมอาจเกินช่วงของ int
ครอบคลุม: ทศนิยม ค่าลบ ศูนย์ สมาชิกเดียว และค่ามากที่ทำให้ผลรวมเกิน int

Example Input	Expected Result
`{1, 2}`	`return 1.5`
`{-2, -3, -5}`	`return -3.333333333...`
`{0, 0, 0}`	`return 0.0`
`{INT_MAX, INT_MAX}`	`return (double)INT_MAX`

รัน: ./array_lab average | 10 เทส x 2 คะแนน = 20 คะแนน

เรียงข้อมูลและกลับข้อความ

Expected Result คือข้อมูลหลังเรียกฟังก์ชัน ไม่ใช่ข้อความที่ต้อง printf

03 เรียงจากมากไปน้อยด้วย Bubble Sort | 20 คะแนน

```
void sort_descending(int a[], int n)
```

n >= 1; ใช้ Bubble Sort แก้ค่าใน a เดิม; เก็บค่าซ้ำให้ครบ
ครอบคลุม: เรียงแล้ว เรียงย้อนกลับ ค่าซ้ำ int limits และอาเรย์ 64 สมาชิก
เทสตรวจผลลัพธ์; ผู้สอนตรวจว่ามีการใช้ Bubble Sort จากโค้ดอีกครั้ง

Example Input	Expected Result
`{3, 1, 4, 2}`	`a = {4, 3, 2, 1}`
`{3, 1, 3, 2, 1}`	`a = {3, 3, 2, 1, 1}`
`{-2, 0, -7, 4}`	`a = {4, 0, -2, -7}`
`{-1, 2}`	`a = {2, -1}`

รัน: ./array_lab sort | 10 เทส x 2 คะแนน = 20 คะแนน

04 กลับลำดับตัวอักษร (Reverse Word) | 20 คะแนน

```
void reverse_word(char word[])
```

word เป็น ASCII ที่มี \0 ปิดท้าย; กลับข้อความในอาเรย์เดิมและรักษา \0
ใช้ strlen ได้ ห้ามใช้ strrev; เทสมีข้อความยาว 0 ถึง 63 ตัวอักษร
ครอบคลุม: ว่าง อักษรเดียว ความยาวคู่/คี่ คำสมมาตร อักษรซ้ำ และช่องว่าง

Example Input	Expected Result
`"HELLO"`	`word = "OLLEH"`
`"ABCD"`	`word = "DCBA"`
`""`	`word = ""`
`"A"`	`word = "A"`
`" ab "`	`word = " ba "`

รัน: ./array_lab reverse | 10 เทส x 2 คะแนน = 20 คะแนน

ข้อควรสังเกต

ช่องว่างเป็นส่วนหนึ่งของ String ต้องคงอยู่ครบ; ข้อนี้ไม่ทดสอบข้อความภาษาไทย UTF-8

อาเรย์ 2 มิติและการส่งงาน

รวม 50 เทส • เทสละ 2 คะแนน • คะแนนเต็ม 100

05 หาผลรวมของแต่ละแถว | 20 คะแนน

```
void row_sums(int rows, int matrix[rows][3], int sums[])
```

rows >= 1; มี 3 คอลัมน์เสมอ; เขียนผลรวมแถว r ใน sums[r]; ห้ามแก้ matrix
sums มีพื้นที่ครบแล้ว; ผลรวมแต่ละแถวในเทสอยู่ในช่วงของ int
ครอบคลุม: แถวเดียว ศูนย์ ค่าลบ ค่าสูง/ต่ำของ int และ 8 แถวอิสระ

Example Input	Expected Result
`{{1,2,3}, {4,5,6}}`	`sums = {6, 15}`
`{{-1,-2,-3}, {-4,-5,-6}}`	`sums = {-6, -15}`
`{{0,0,9}, {0,-5,0}, {7,0,0}}`	`sums = {9, -5, 7}`
`{{INT_MAX,0,0}, {INT_MIN,0,0}}`	`sums = {INT_MAX, INT_MIN}`

รัน: ./array_lab rows | 10 เทส x 2 คะแนน = 20 คะแนน

คอมไพล์ใหม่ แล้วตรวจคะแนน

```
gcc -std=c99 -Wall -Wextra -pedantic array_lab.c -o array_lab
```

```
./array_lab          ./array_lab average
```

รันทั้งหมดได้คะแนนเต็ม 100; เลือกข้อเดียวได้คะแนนเต็ม 20 เฉพาะข้อนั้น
FAIL จะแสดง input / expected / actual; ข้อที่ห้ามแก้ข้อมูลจะตรวจว่าค่าเดิมยังอยู่
Windows (GCC): ตั้งชื่อผลลัพธ์ array_lab.exe แล้วรัน ./array_lab.exe

ตัวอย่างคะแนนบางส่วน (สมมติ ไม่ใช่คะแนนเริ่มต้น)

```
2. AVERAGE           : 14/20 points (7/10 tests)
TOTAL SCORE: 14/20 points
RESULT: 7/10 passed
```

ได้ 7 เทส x 2 = 14 คะแนน; เมื่อผ่านทั้งหมดจะแสดง TOTAL SCORE: 100/100 points

ก่อนส่งงาน

ส่ง array_lab.c ที่เติมโค้ดแล้ว โดยไม่แก้ส่วนตรวจหรือเขียนคำตอบตายตัวตามเทส
เทสที่ข้อมูลไม่ต้องเปลี่ยนอาจผ่านแม้ยังไม่ได้ทำ ต้องตรวจโค้ดประกอบคะแนน
หากค้าง ให้กด Ctrl+C แล้วตรวจลูปและ Index; คะแนนสรุปได้เมื่อโปรแกรมรันจนจบ